Functions and Pointers in C

Functions in C

1. Introduction to Functions

- **Definition**: A function is a reusable block of code that performs a specific task.
- **Benefits**: Improves modularity, reusability, and simplifies debugging and maintenance.

2. Types of Functions

a. Predefined Functions

- Provided by libraries.
- Examples:
    - printf() for output
    - scanf() for input
    - strlen() from <string.h> for string length

b. User-Defined Functions

- Defined by the programmer.

Example: User-Defined Function

#include <stdio.h>

```
// Function to calculate the square of a number

int square(int num) {

    return num * num;

}



int main() {

    int num = 5;

    printf("Square of %d is %d\n", num, square(num));

    return 0;

}
```

3. Parameter Passing

Call by Value

- Only a copy of the variable is passed.
- Changes inside the function **do not affect** the original variable.

c

```
#include <stdio.h>

void changeValue(int x) {

    x = 10;
```

```
}


int main() {
    int num = 5;
    changeValue(num);
    printf("Value of num: %d\n", num); // Outputs: 5
    return 0;
}
```

Call by Reference

- The actual address of the variable is passed.
- Changes inside the function **affect** the original variable.

```
#include <stdio.h>
void changeValue(int *x) {
    *x = 10;
}


int main() {
    int num = 5;
```

```
    changeValue(&num);

    printf("Value of num: %d\n", num); // Outputs: 10

    return 0;

}
```

4. Recursion

- A function that calls itself.
- **Use Cases**: Factorial calculation, Fibonacci series, and Tower of Hanoi.

Example: Factorial Using Recursion

```
#include <stdio.h>

int factorial(int n) {

    if (n == 0) return 1; // Base case

    return n * factorial(n - 1); // Recursive case

}


int main() {

    int num = 5;

    printf("Factorial of %d is %d\n", num, factorial(num));
```

```
    return 0;

}
```

What Are Pointers in C?

A **pointer** is a variable that stores the memory address of another variable. Instead of holding a value directly, it points to the location where the value is stored.

Why Do We Need Pointers?

1. **Efficient Memory Access**: Helps in directly accessing and modifying variables in memory.
    - Example: Passing large data structures like arrays to functions without copying them.
2. **Dynamic Memory Allocation**: Allows allocating memory at runtime.
    - Example: Allocating memory for arrays when their size isn't known during compilation.
3. **Advanced Data Structures**: Used to create structures like linked lists, trees, and graphs.
4. **Call by Reference**: Enables modifying variables directly in functions.

Pointer Basics

Declaration:
A pointer is declared using the * symbol.
Syntax:
c

```
data_type *pointer_name;
```

Example:
c

```
int *ptr; // Declares a pointer to an integer
```

Initialization:
Assign the address of a variable using the & operator.
Example:
c

```
int num = 10;

int *ptr = &num; // 'ptr' stores the address of 'num'
```

Dereferencing:
Access or modify the value at the address stored in the pointer using the * operator.
Example:
c

```
printf("%d", *ptr); // Outputs the value of 'num'
```

Memory Representation

Suppose num is stored at memory address 2000.

- ptr holds the value 2000 (address of num).
- *ptr gives the value stored at 2000, which is 10.

Example 1: Basic Pointer Operations

c

```
#include <stdio.h>
```

```
int main() {
    int num = 10;
    int *ptr = &num; // Pointer initialized with address of 'num'

    printf("Address of num: %p\n", ptr); // Prints address of 'num'
    printf("Value of num using pointer: %d\n", *ptr); // Dereference to get value

    // Modify value using pointer
    *ptr = 20;
    printf("Modified value of num: %d\n", num);

    return 0;
}
```

Output:

Address of num: 0x7ffee37fbc44 (example memory address)

Value of num using pointer: 10

Modified value of num: 20

Example 2: Call by Reference Using Pointers

c

```
#include <stdio.h>

void swap(int *x, int *y) {
    int temp = *x;
    *x = *y;
    *y = temp;
}

int main() {
    int a = 5, b = 10;
    printf("Before swap: a = %d, b = %d\n", a, b);

    swap(&a, &b); // Passing addresses of 'a' and 'b'
    printf("After swap: a = %d, b = %d\n", a, b);

    return 0;
}
```

Output:

Before swap: a = 5, b = 10

After swap: a = 10, b = 5

Example 3: Pointers and Arrays

c

```
#include <stdio.h>

int main() {
    int arr[3] = {10, 20, 30};
    int *ptr = arr; // Pointer to the first element of the array

    for (int i = 0; i < 3; i++) {
        printf("Value at index %d: %d\n", i, *(ptr + i)); // Access array elements using pointer
    }
```

```
    return 0;

}
```

Output:

```
Value at index 0: 10

Value at index 1: 20

Value at index 2: 30
```

Key Points

Declaration:
c

```
int *ptr;  // Pointer to an integer
```

1. **Initialization**:

   ```
   int num = 10;

ptr = &num; // Pointer stores address of 'num'
```

2. **Dereferencing**:

   ```
   *ptr = 20; // Modifies the value at the address stored in 'ptr'
   ```

3.

Why We Use Pointers?

1. **Memory Efficiency**: Reduces memory usage by avoiding data duplication.
2. **Dynamic Programming**: Enables complex data structures and dynamic memory allocation.
3. **Flexibility**: Simplifies handling arrays and strings, and enhances function capabilities.

Pointers are a cornerstone of programming and open the door to advanced techniques like dynamic memory allocation and data structures.

Explanation of int *p = # in Detail

This statement involves three key elements: **pointer declaration**, **pointer initialization**, and **address-of operator**.

1. Variable num

int num;

Here:

- num is an **integer variable**.
- It is allocated a memory location in RAM (let's assume its address is 2000 for explanation purposes).
- If num = 10;, the value 10 is stored at memory address 2000.

---

2. Pointer Declaration

```
int *p;
```

Here:

- p is declared as a **pointer to an integer**.
- The * in int *p indicates that p is not a regular variable but a **pointer variable** that will store the **memory address** of an integer.
- At this stage, p is uninitialized, meaning it does not point to any valid memory.

3. Address-of Operator (&)

```
p = &num;
```

Here:

- The & operator is the **address-of operator**. It retrieves the **memory address** of a variable.
- In this case, &num fetches the memory address of num (e.g., 2000).
- Now, p stores the address of num.

4. Combining It All

```
int *p = &num;
```

- **int *p**: Declares a pointer p that can store the address of an integer.
- **&num**: Fetches the memory address of num.
- **p = #**: Assigns the address of num to the pointer p.

After execution:

- p contains the memory address of num (e.g., 2000).
- Dereferencing p using *p will give the value stored at that memory address (i.e., 10).

Memory Representation

Variable	Value	Address
num	10	2000
p	2000	3000 (example pointer storage address)

Practical Example

```
#include <stdio.h>

int main() {
```

```
    int num = 10;    // Declare an integer variable and assign value 10

    int *p = &num;   // Declare a pointer and assign the address of 'num'


    printf("Address of num: %p\n", &num); // Prints the address of 'num'

    printf("Value of p (address of num): %p\n", p); // Prints what 'p' stores

    printf("Value pointed by p: %d\n", *p); // Dereferences 'p' to get the value of 'num'


    // Modify value of num using pointer

    *p = 20;

    printf("Updated value of num: %d\n", num);


    return 0;

}
```

Output

Address of num: 0x7ffee17ab44c (example memory address)

Value of p (address of num): 0x7ffee17ab44c

Value pointed by p: 10

Updated value of num: 20

Summary

1. **int *p** declares a pointer variable p that can store the address of an integer.
2. **&num** gets the memory address of num.
3. **p = &num** assigns the address of num to p.
4. ***p** dereferences the pointer to access or modify the value of num.

Pointers are powerful because they allow indirect access to variables, which is useful for dynamifc memory allocation, arrays, and efficient data handling.

2. Types of Pointers

a. Void Pointer

- A pointer without a specific data type.
- Can point to any data type.

c

```
void *ptr;

int num = 5;

ptr = &num; // Valid
```

b. Pointer to Pointer

- Stores the address of another pointer.

c

```
int num = 10;

int *p = &num;

int **pp = &p; // Pointer to pointer
```

3. Pointer Arithmetic

- You can perform operations like addition and subtraction on pointers.

c

```
int arr[] = {10, 20, 30};

int *p = arr;

printf("%d\n", *(p + 1)); // Outputs: 20
```

4. Parameter Passing Using Pointers

a. Call by Value

- Passing a copy of the variable.

b. Call by Reference

- Passing the address using pointers.

Programs for Practice

Finding Maximum Using Pointers

c

```
#include <stdio.h>

void findMax(int *a, int *b, int *max) {
    *max = (*a > *b) ? *a : *b;
}

int main() {
    int num1 = 10, num2 = 20, max;
    findMax(&num1, &num2, &max);
    printf("Maximum: %d\n", max);
    return 0;
}
```

Swapping Two Numbers Using Pointers

```
#include <stdio.h>

void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

int main() {
```

```
    int x = 5, y = 10;

    swap(&x, &y);

    printf("x: %d, y: %d\n", x, y); // Outputs: x: 10, y: 5

    return 0;

}
```